

WEEK 7 — VAPT

WEB APPLICATION SECURITY TESTING WITH BURP SUITE

PRACTICAL ASSESSMENT Reconnaissance • HTTP Interception • Manual Vulnerability Testing • Endpoint Discovery • Professional VAPT Reporting

Document Field	Details
Prepared By	Krina Sorathiya
Training Module	Week 7 — Vulnerability Assessment & Penetration Testing
Primary Tool	Burp Suite Community Edition
Practice Environment	DVWA / locally hosted authorized laboratory
Supporting Tools	OWASP ZAP, Gobuster / Dirsearch
Assessment Type	Web Application Security Testing
Submission	12–15 page report, 8–10 slide PPT, LinkedIn learning post
Deadline	21 September

CONFIDENTIAL — TRAINING / AUTHORIZED LAB USE

1. Introduction to VAPT

Vulnerability Assessment and Penetration Testing (VAPT) is a structured process used to identify, validate, document, and help remediate security weaknesses in an information system. Unlike a basic vulnerability scan, a practical penetration test attempts to validate whether a suspected weakness can actually influence application behavior within an authorized scope.

For web applications, VAPT requires understanding more than individual vulnerabilities. A tester must understand how browsers communicate with servers, how HTTP requests are constructed, how authentication and sessions work, how user input is processed, how authorization decisions are made, and how application functionality can be abused when security controls are missing or incorrectly implemented.

This Week 7 exercise focuses on the practical application of these principles using Burp Suite against intentionally vulnerable local applications. The assessment is designed to demonstrate a complete workflow from reconnaissance to evidence-backed reporting.

1.1 Objectives of the Assessment

- Understand the role of an interception proxy in web application testing.
- Configure Burp Suite and a browser for controlled HTTP/HTTPS interception.
- Perform reconnaissance and map the application's accessible attack surface.
- Identify input points, parameters, cookies, authentication flows, and interesting endpoints.
- Validate common web vulnerabilities using controlled test cases.
- Use Burp Repeater and Intruder for repeatable request manipulation in the lab.
- Perform endpoint discovery with Burp, Gobuster, or Dirsearch.
- Create professional findings containing evidence, impact, root cause, and remediation.

LAB SAFETY All testing described in this report is restricted to intentionally vulnerable applications and systems for which testing authorization exists. The techniques should not be applied to public or third-party systems without explicit permission.

2. Web Application Security Fundamentals

A web application normally consists of a client, a web server, application logic, and one or more backend services such as a database. The browser sends HTTP requests containing a method, URL, headers, cookies, and sometimes a body. The server processes the request and returns an HTTP response containing a status code, headers, and response content.

From a security-testing perspective, every element controlled by the client must be treated as potentially untrusted. This includes query parameters, form fields, JSON values, cookies, HTTP headers, uploaded filenames, and hidden form fields. A secure application must validate authorization and business rules on the server instead of assuming that values received from the browser are trustworthy.

2.1 HTTP Request Anatomy

Component	Security Relevance
Method	GET, POST, PUT, DELETE and other methods can expose different functionality and validation paths.
URL / Path	Identifies the application resource being requested.
Query Parameters	Common input locations for filtering, searching, IDs, pagination, and application logic.
Headers	May influence content negotiation, authentication, caching, origin behavior, and other controls.
Cookies	Often contain session identifiers and other client state; their security attributes matter.
Request Body	Can contain form data, JSON, XML, file metadata, or other user-controlled values.

2.2 HTTP Response Anatomy

The response contains a status code, headers, and a body. During testing, the response can reveal whether an input was accepted, rejected, reflected, stored, redirected, or processed differently. Differences in response length, status code, content, redirects, and error messages can be useful indicators during manual validation.

3. Burp Suite — Role in Web Application Testing

Burp Suite is an integrated platform for security testing of web applications. Its proxy functionality places Burp between the browser and the target server, allowing the tester to inspect and, where appropriate, modify requests before forwarding them.

This interception capability is important because a normal browser interface hides many of the details that matter to a security tester. Burp exposes parameters, cookies, headers, methods, response codes, redirects, and application paths in a form that can be reviewed and manipulated in a controlled manner.

Burp Component	Purpose in Week 7
Proxy	Intercept, inspect, and forward HTTP/HTTPS requests.
HTTP History	Review traffic generated during application browsing.
Target / Site Map	Organize discovered application resources and attack surface.
Repeater	Manually resend and modify requests for validation.
Intruder	Perform controlled automated variations of selected request parameters.
Inspector	Review structured request and response elements.

3.1 Proxy Architecture

The testing architecture used in this exercise is conceptually: Browser → Burp Proxy (127.0.0.1:8080) → Local DVWA → Application/Database. The browser generates the request, Burp receives it first, and the tester can inspect the request before forwarding it to the application.

Burp

Project

Intruder

Repeater

View

Help

Burp Suite Community Edition v2024.7.1 - Temporary Project

Settings

Dashboard

Target

Proxy

Intruder

Repeater

Collaborator

Sequencer

Decoder

Comparer

Logger

Organizer

Extensions

Learn

Intercept

HTTP history

WebSockets history

Match and replace

Proxy settings

Search

General

Proxy

TLS

HTTP

Match and replace

Miscellaneous

?

Proxy listeners

Burp Proxy uses listeners to receive incoming HTTP requests from your browser. You will need to configure your browser to use one of the listeners as its proxy server.

Add

Edit

Remove

Running	Interface	Invisible	Redirect	Certificate	TLS Protocols
☒	127.0.0.1:8080	☐		Per-host	Default

Each installation of Burp generates its own CA certificate that Proxy listeners can use when negotiating TLS connections. You can import or export this certificate for use in other tools or another installation of Burp.

Import / export CA certificate

Regenerate CA certificate

?

Intercept client requests

Use these settings to control which requests are stalled for viewing and editing in the Intercept tab.

☒ Intercept client requests

I intercept requests based on the following rules:

Add

Edit

Remove

Up

Down

Enabled	Operator	Match type	Relationship	Condition
☒		File extension	Does not match	(^gif\$ ^jpg\$ ^png\$ ^css\$ ^js\$ ^ico\$ ^svg\$ ^woff\$ ^ttf\$ ^eot\$ ^...

☒ Automatically fix missing or superfluous new lines at end of request

☒ Automatically update Content-Length header when the request is edited

Event log (1)

All issues

Memory: 129.5MB

4. Task 22 — Burp Suite Setup & Interception

The first practical task is to establish a reliable interception environment. A security tester should be able to capture ordinary application traffic before attempting vulnerability testing because request interception provides the baseline information required for later manipulation.

4.1 Browser Proxy Configuration

The browser is configured to use the Burp listener at 127.0.0.1:8080. This causes requests generated by the browser to pass through Burp. In a dedicated training browser profile, the proxy configuration should be used only while testing the authorized laboratory so that unrelated browsing traffic is not captured.

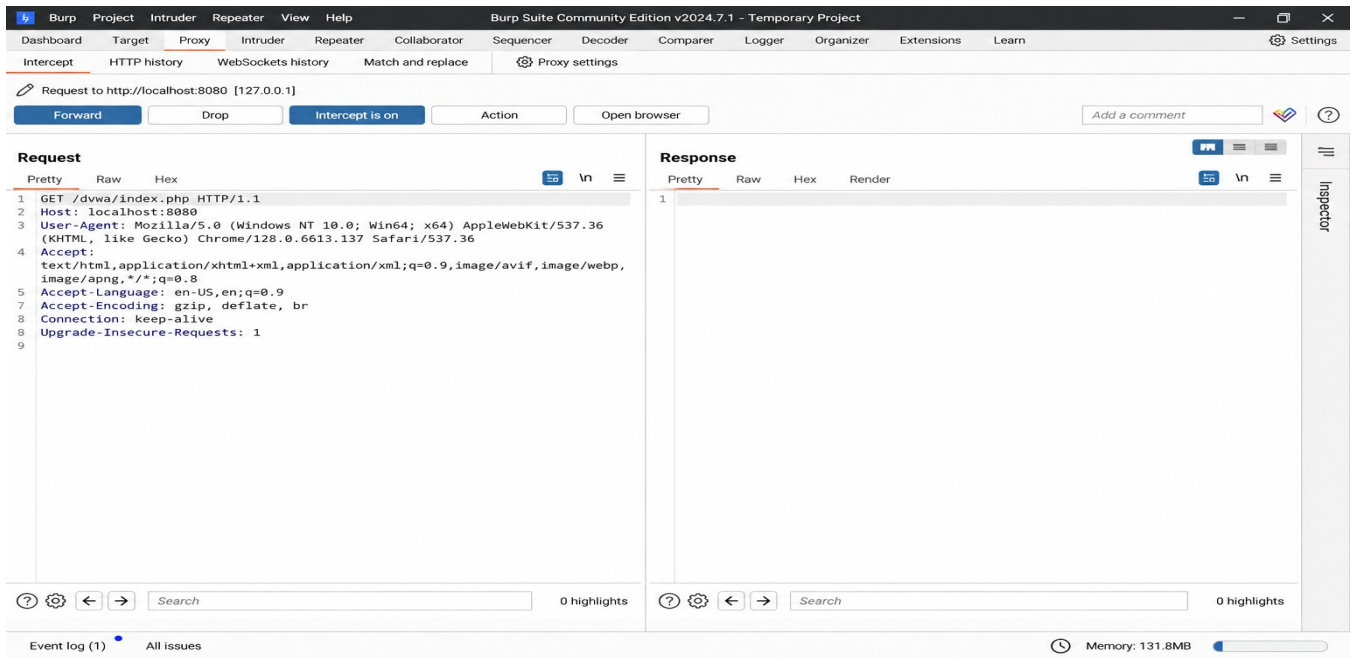
4.2 HTTPS Certificate

HTTPS encrypts traffic between the browser and server. In an authorized lab, Burp can generate a local Certificate Authority certificate that the dedicated browser profile trusts. This permits Burp to decrypt and re-encrypt the HTTPS traffic for inspection. The certificate should never be installed as a trusted authority in an unmanaged or personal environment without understanding the security implications.

4.3 Interception Workflow

1. Start the authorized DVWA laboratory.
2. Launch Burp Suite Community Edition.
3. Verify the Proxy listener on 127.0.0.1:8080.
4. Configure the dedicated browser to use the local proxy.
5. Enable interception and browse to the DVWA login page.
6. Capture a normal request and inspect the method, path, headers, cookies, and body.
7. Forward the request and compare the application response.

HTTPS / CERTIFICATE CONFIGURATION



5.Reconnaissance & Application Mapping

Reconnaissance is the information-gathering stage of a VAPT assessment. In web application testing, the objective is to understand what functionality exists before attempting to exploit it. Good reconnaissance reduces blind testing and helps the tester identify relationships between pages, parameters, sessions, and application workflows.

Burp's Target/Site Map provides a visual representation of discovered paths. Passive mapping occurs naturally as the tester browses the application. The resulting map can include pages, scripts, APIs, forms, static resources, and endpoints.

5.1 What to Identify

- Authentication and logout endpoints.
- Password reset or account recovery functionality where present.
- Search, profile, comment, upload, and other user-input forms.
- GET and POST parameters.
- Numeric identifiers and object references.
- Session cookies and other state-management values.
- Administrative or privileged paths.
- API endpoints and JavaScript-referenced routes.
- Error pages and unusual response behavior.

5.2 Attack Surface Concept

The attack surface is the collection of reachable components through which an attacker could interact with the application. A larger attack surface does not automatically mean a vulnerability exists, but every input or function may require appropriate authentication, authorization, validation, and output encoding.

BURP TARGET / SITE MAP

⚙️

Settings

🔍 Search

General

Proxy

TLS

HTTP

Match and replace

Miscellaneous

ⓘ

TLS

These settings control how Burp handles TLS connections.

Client TLS handling

Burp can act as a TLS client when making outgoing connections. You can configure how Burp handles TLS certificates here.

☒ Use system TLS settings

☒ Enable TLS hostname verification

Server TLS certificate

Burp generates its own CA certificate to sign certificates for proxied HTTPS connections. You can import or export this certificate for use in other tools or another installation of Burp.

Import / export CA certificate

Regenerate CA certificate

Intercept HTTPS connections

Use these settings to control which HTTPS connections are stalled for viewing and editing in the Intercept tab.

☒ Intercept HTTPS connections

Add

Edit

Remove

Enabled	Host / IP range	Port	Action
☒	All	All	Intercept

Event log (3)

All issues

⌚

Memory: 128.9MB

Burp Suite Community Edition

Not secure | http://burp/cert

Burp Suite Community Edition

CA Certificate

Use this certificate to configure your browser or other tools to trust Burp as a certificate authority. This allows Burp to intercept and decrypt HTTPS traffic.

Download CA certificate

←

⚙️

Certificate Import Wizard

×

Certificate Store

Certificate stores are system areas where certificates are kept.

Windows can automatically select a certificate store, or you can specify a location for the certificate.

☐ Automatically select the certificate store based on the type of certificate

☒ Place all certificates in the following store

Certificate store:

Trusted Root Certification Authorities

Browse...

Next

Cancel

🏠

Type here to search

📅

🌐

📁

🔍

📧

📺

🔧

🔥

🔌

⬆️

🇬🇧

ENG

11:24 AM

15-09-2026

6. Authentication & Session Analysis

Authentication verifies the identity of a user, while authorization determines what an authenticated user is allowed to do. These controls are related but distinct. A user may successfully authenticate and still be unauthorized to access an administrative function.

During login analysis, the tester should examine the request method, credential parameters, response status, redirects, session cookies, and behavior after logout. Testing should remain within the provided training credentials and must not involve real user accounts.

Area	Questions for the Tester
Credentials	Are credential parameters sent in the expected request body?
Transport	Is the lab using an appropriate protected transport when required?
Session	Does successful login establish or rotate a session identifier?
Cookie Security	Are HttpOnly, Secure, and SameSite attributes configured appropriately?
Logout	Does logout invalidate the session as expected?
Errors	Do failure messages reveal unnecessary account information?

CAPTURED LOGIN REQUEST

⚡ Burp Project Intruder Repeater View Help

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn

Site map Scope Issue definitions

Filter: Hiding not found items; hiding CSS, image and general binary content; hiding 4xx responses; hiding empty folders

⌵ http://localhost:8080

/

dvwa

css

images

includes

js

vulnerabilities

brute

captcha

csrf

exec

fi

id

sql

sql_inject

upload

xss

xss_s

DVWA

index.php

login.php

logout.php

security.php

setup.php

external

hackable

phpids

suggestions

favicon.ico

robots.txt

Host	Method	URL	Params	Status code	Length	MIME type	Title
http://localhost:8080	GET	/		200	5370	HTML	Vulnerability: Welcome to ..
http://localhost:8080	GET	/dvwa/		302	554	HTML	Found
http://localhost:8080	GET	/dvwa/index.php		200	5370	HTML	Vulnerability: Welcome to ..
http://localhost:8080	GET	/dvwa/login.php		200	4208	HTML	Login :: Damn Vulnerable ..
http://localhost:8080	GET	/dvwa/security.php		200	5048	HTML	DVWA Security :: Damn V...
http://localhost:8080	GET	/dvwa/vulnerabilities/sql/		200	4861	HTML	Vulnerability: SQL Injectio...
http://localhost:8080	GET	/dvwa/vulnerabilities/xss_r/		200	4789	HTML	Vulnerability: Reflected Cr...
http://localhost:8080	GET	/dvwa/vulnerabilities/xss_s/		200	4871	HTML	Vulnerability: Stored Cros...
http://localhost:8080	GET	/dvwa/vulnerabilities/upload/		200	5098	HTML	Vulnerability: File Upload ...

Request

Pretty Raw Hex

1 GET /dvwa/index.php HTTP/1.1

2 Host: localhost:8080

3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.6613.137 Safari/537.36

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.9

6 Accept-Encoding: gzip, deflate, br

7 Connection: keep-alive

8 Upgrade-Insecure-Requests: 1

9

0 highlights

Response

Pretty Raw Hex Render

1 HTTP/1.1 200 OK

2 Date: Mon, 15 Sep 2026 05:52:10 GMT

3 Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12

4 X-Powered-By: PHP/8.2.12

5 Set-Cookie: PHPSESSID=5h9q2m3k8v117n4p6r0s2t9u8; path=

6 Expires: Thu, 19 Nov 1981 08:52:00 GMT

7 Cache-Control: no-store, no-cache, must-revalidate

8 Pragma: no-cache

9 Content-Length: 5370

10 Content-Type: text/html; charset=UTF-8

11

12 <!DOCTYPE html>

13 <html lang="en-GB">

14 <head>

15 <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />

16 <title>

17 Vulnerability: Welcome to Damn Vulnerable Web Application

18 </title>

0 highlights

Event log (3) All issues

Memory: 126.1MB

11:32 AM 15-09-2026

7. Task 24 — Manual Vulnerability Testing

Manual vulnerability testing is the process of intentionally modifying controlled inputs to determine whether the application behaves securely. Burp Repeater is especially useful because it allows the tester to preserve a captured request and repeatedly test one change at a time.

The key principle is comparison. A tester should first understand the baseline request and response, then introduce one controlled change and compare the resulting behavior. This makes the evidence more defensible and reduces confusion caused by multiple simultaneous modifications.

7.1 SQL Injection — Theory

SQL Injection occurs when an application constructs a database query using untrusted input in a way that changes the intended SQL statement. If a vulnerable query expects a numeric identifier, for example, but directly concatenates the supplied value into SQL, specially crafted input may alter the query's logical condition.

The secure design is to use parameterized queries or prepared statements. In this model, user input is treated as data rather than executable SQL syntax. Input validation and least-privilege database accounts provide additional layers of protection.

For the DVWA lab, a controlled boolean-style input such as ' OR 1=1 -- can be used solely to demonstrate changed query behavior. The purpose is to prove the weakness, not to extract unrelated data.

SQL INJECTION REQUEST

Burp Suite Community Edition v2024.7.1 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings

1 x 2 x 3 x +

Send Cancel < >

Target: http://localhost:8080 HTTP/1

Request

Pretty Raw Hex

```
1 POST /dvwa/vulnerabilities/sqli/ HTTP/1.1
2 Host: localhost:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
  AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.6613.137
  Safari/537.36
4 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
  image/apng,*/*;q=0.8
5 Content-Type: application/x-www-form-urlencoded
6 Content-Length: 43
7 Origin: http://localhost:8080
8 Connection: keep-alive
9 Referer: http://localhost:8080/dvwa/vulnerabilities/sqli/
10 Cookie: security=low; PHPSESSID=7f3a2c8d9e5b1a4d8c9f0e3b2a1c6d4e
11 Upgrade-Insecure-Requests: 1
12
13 id=1' OR '1'='1&Submit=Submit
```

Response

Pretty Raw Hex Render

DVWA

Vulnerability: SQL Injection

User ID:

ID: 1' OR '1'='1
First name: admin
Surname: admin

ID: 1' OR '1'='1
First name: Gordon
Surname: Brown

ID: 1' OR '1'='1
First name: Hack
Surname: Me

ID: 1' OR '1'='1
First name: Pablo
Surname: Picasso

ID: 1' OR '1'='1
First name: Bob
Surname: Smith

More Information

Home
Instructions
Setup
Brute Force
Command Execution
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass

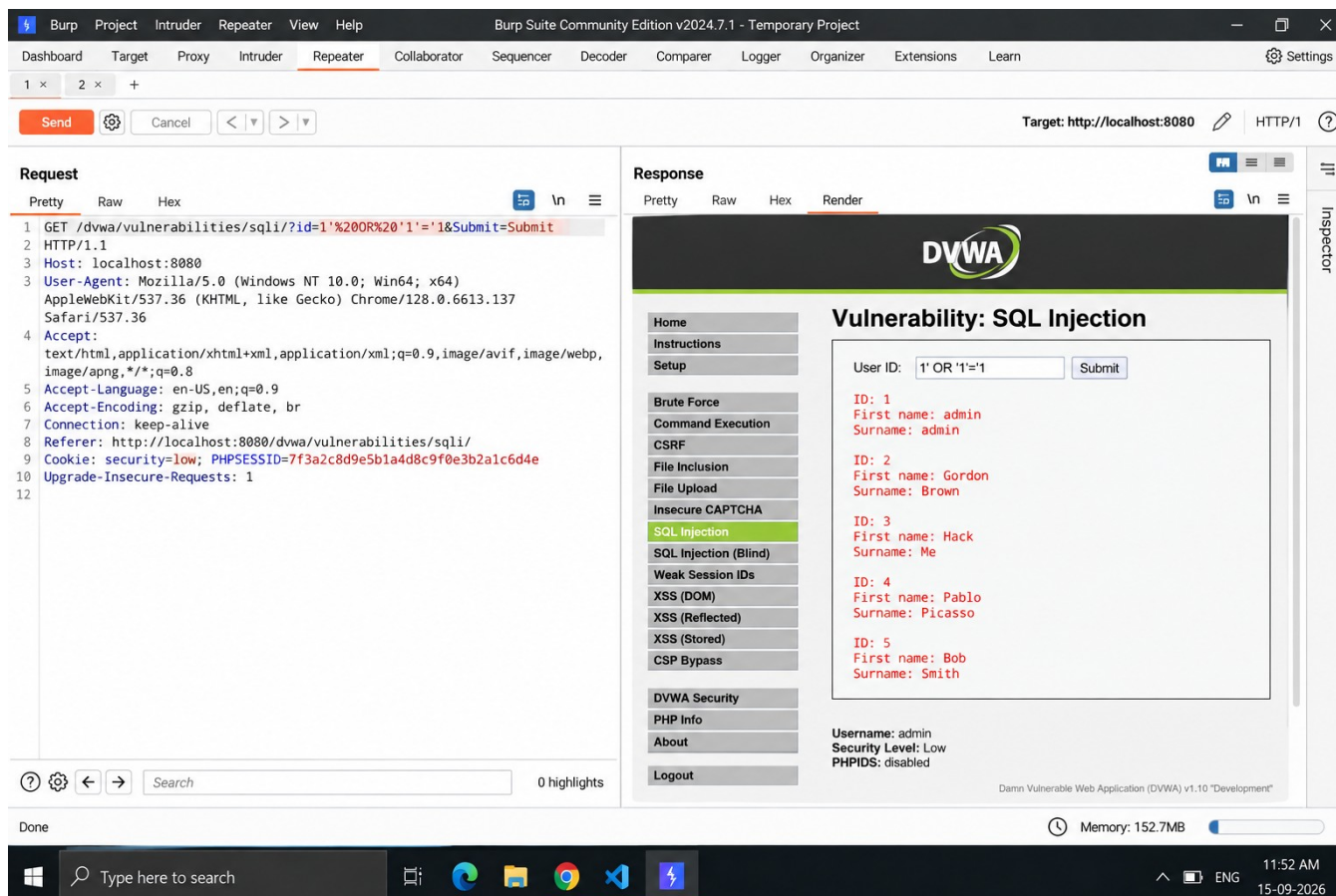
0 highlights

Memory: 133.2MB

Type here to search

11:45 AM
15-09-2026

SQL INJECTION RESULT



SQL INJECTION RESULT

8. Cross-Site Scripting — Stored & Reflected

Cross-Site Scripting occurs when an application places untrusted data into a browser-executable context without applying the required output encoding or sanitization. XSS is primarily a client-side execution problem, but its root cause is frequently an application-side failure to correctly handle untrusted input.

8.1 Reflected XSS

In reflected XSS, the malicious input is supplied in a request and immediately returned in the response. A victim generally needs to interact with a crafted URL or request for the payload to execute. The risk depends on the affected page, victim privileges, browser context, and how the reflected value is used.

8.2 Stored XSS

In stored XSS, the application saves attacker-controlled content and later displays it to users. Because the payload can affect multiple visitors, stored XSS can have broader impact than a one-time reflected injection.

8.3 Safe Lab Proof

A harmless proof such as `<script>alert(1)</script>` is sufficient for the DVWA exercise. The alert demonstrates script execution without attempting to steal cookies, credentials, or sensitive information.

8.4 XSS Prevention

- Perform context-aware output encoding.
- Validate input against expected application rules.
- Use safe DOM APIs and avoid unsafe HTML insertion where possible.
- Deploy Content Security Policy as defense in depth.
- Protect session cookies using appropriate security attributes.

STORED XSS INPUT & OUTPUT

The image is a composite of three screenshots illustrating a Stored XSS attack on DVWA.

Top Left: DVWA Vulnerability: Stored Cross Site Scripting (XSS)
The interface shows the 'XSS (Stored)' tab selected in the left sidebar. The 'Name' field contains `<script>alert('XSS')</script>` and the 'Message' field contains 'This is a test for stored XSS'. A 'Sign Guestbook' button is visible.

Top Right: Browser Alert
A modal dialog box from 'localhost:8080' says 'XSS' with an 'OK' button.

Bottom: Burp Suite Network Traffic
The Burp Suite interface shows a request and response for the XSS payload.

1. Input Payload (Stored XSS)
The 'Request' tab shows the following details:
Method: POST
URL: /dvwa/vulnerabilities/xss_s/
Host: localhost:8080
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Content-Type: application/x-www-form-urlencoded
Content-Length: 86
Origin: http://localhost:8080
Referer: http://localhost:8080/dvwa/vulnerabilities/xss_s/
Cookie: security=low; PHPSESSID=7f3a2c8d99e5b1a4d8c9f0e3b2a1c6d4e
Connection: keep-alive
Body: `txtName=<script>alert('XSS')</script>&mtxMessage=This+is+a+test+for+stored+XSS.&btnSign=Sign+Guestbook`

2. Result (Stored XSS - Browser Alert)
The 'Response' tab shows the following details:
Status: 200
Date: Mon, 15 Sep 2026 06:12:45 GMT
Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
X-Powered-By: PHP/8.2.12
Location: xss_s.php
Content-Length: 0
Connection: close
Content-Type: text/html; charset=UTF-8

3. Burp Request (Stored XSS) Payload
The request body is: `txtName=<script>alert('XSS')</script>&mtxMessage=This+is+a+test+for+stored+XSS.&btnSign=Sign+Guestbook`

4. Burp Response (Redirect after Submission)
The response status is 200 and the location is `xss_s.php`.

REFLECTED XSS INPUT & OUTPUT

1. Reflected XSS Input (DVWA Application)

The DVWA application interface shows the 'Vulnerability: Reflected Cross Site Scripting (XSS)' section. The 'What's your name?' input field contains the payload: `<script>alert('XSS')</script>`. The 'Submit' button is visible.

2. Reflected XSS Output (Browser Alert)

A browser alert box is displayed, showing the message: 'localhost:8080 says XSS'. The alert box has an 'OK' button.

3. Burp Request (Reflected XSS Payload)

The Burp Suite interface shows the intercepted request to `http://localhost:8080`. The request details are as follows:

```
1 GET /dwa/vulnerabilities/xss_r/?name=<script>alert('XSS')</script> HTTP/1.1
2 Host: localhost:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.96613.137 Safari/537.36
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.9
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
```

4. URL with Payload (After Submission)

The browser address bar shows the URL after submission: `http://localhost:8080/dwa/vulnerabilities/xss_r/?name=<script>alert('XSS')</script>`.

9. Authentication Testing & Parameter Tampering

Authentication testing evaluates whether the application correctly handles identity verification. In an intentionally vulnerable lab, the tester may examine weak or default credentials provided by the application, login error behavior, session handling, and access restrictions. Testing should never move beyond the authorized account and environment.

Parameter tampering occurs when a client-controlled value is modified in an attempt to influence application logic. Examples include changing an object ID, quantity, role value, price, or other parameter. The central security requirement is that the server must independently validate authorization and business rules rather than trusting the client's value.

Scenario	Expected Secure Behavior
Modified object ID	Server checks whether the current user is authorized for the referenced object.
Modified role value	Server ignores or rejects unauthorized privilege changes.
Modified business value	Server recalculates trusted values instead of accepting client-controlled totals.
Modified hidden field	Server treats hidden fields as untrusted input.

PARAMETER TAMPERING IN BURP

The screenshot displays the Burp Suite Repeater tab, comparing an original HTTP request with a tampered version. The original request is a GET to `/dvwa/vulnerabilities/id/?id=1&Submit=Submit`. The tampered request changes the `id` parameter to `2`. The responses show that the server returns different user information based on the tampered ID.

Field	Original Request (Before Tampering)	Modified Request (Parameter Tampering)
Request URL	<code>GET /dvwa/vulnerabilities/id/?id=1&Submit=Submit HTTP/1.1</code>	<code>GET /dvwa/vulnerabilities/id/?id=2&Submit=Submit HTTP/1.1</code>
Response User ID	<code><h2>User ID: 1</h2></code>	<code><h2>User ID: 2</h2></code>
Response Last Name	<code><p>Last name: admin</p></code>	<code><p>Last name: Brown</p></code>

10. Burp Repeater & Intruder

Repeater is primarily a manual validation tool. A tester can capture a request, send it to Repeater, change one parameter, resend it, and compare the response. This makes it useful for SQL injection validation, access-control checks, parameter tampering, header manipulation, and input-validation testing.

Intruder is designed for controlled automated request variations. In the training environment, a small payload set can be used to demonstrate how a selected parameter behaves across multiple inputs. Because automated requests can create significant traffic, Intruder should be restricted to the local authorized target and used conservatively.

Tool	Typical Week 7 Use
Repeater	Baseline vs modified request comparison.
Intruder	Small controlled payload variations.
Proxy	Initial interception and traffic inspection.
Target	Application mapping.
HTTP History	Review previously captured traffic.

BURP REPEATER

The screenshot displays the Burp Suite Repeater interface for a target at `http://localhost:8080`. It shows four sequential steps in the Repeater list:

- 1. Original Request (Valid Credentials):** A POST request to `/dvwa/login.php` with headers including `Host: localhost:8080`, `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.6613.137 Safari/537.36`, and a body containing `username=admin&password=password&Login=Login`.
- 2. Response (Successful Login):** The corresponding HTTP response from the DVWA application, showing a successful login message.
- 3. Modified Request (Parameter Tampering):** A modified POST request where the password is changed to `test` in the body: `username=admin' OR '1'='1&password=test&Login=Login`.
- 4. Response (Login Failed):** The response to the tampered request, showing a "Login failed" error message.

BURP INTRUDER

1 x

2 x

+

Positions

Payloads

Resource pool

Settings

?

Choose an attack type

Sniper

Start attack

?

Payload positions

Configure the positions where payloads will be inserted, they can be added into the target as well as the base request.

1 POST /dvwa/login.php HTTP/1.1

2 Host: localhost:8080

3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/128.0.6613.137 Safari/537.36

4 Content-Type: application/x-www-form-urlencoded

5 Content-Length: 43

6 Origin: http://localhost:8080

7 Referer: http://localhost:8080/dvwa/login.php

8 Cookie: security=low; PHPSESSID=7f3a2c8d9e5b1a4d8c9f0e3b2a1c6d4e

9 Connection: keep-alive

10 Upgrade-Insecure-Requests: 1

11

12 username=admin&password=\$password\$&Login=Login

?

Settings

Search

0 highlights

Clear

?

Payload sets

You can define one or more payload sets. The number of payload sets depends on the attack type defined in the Positions tab. Various payload types are available for each payload set, and each payload type can be customized in different ways.

Payload set: 1

Payload count: 4

Payload type: Simple list

Request count: 4

?

Payload options [Simple list]

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load ...

Remove

Clear

Deduplicate

123456

password

admin

letmein

Windows

Type here to search

Taskbar icons

System tray

11:35 AM

15-09-2026

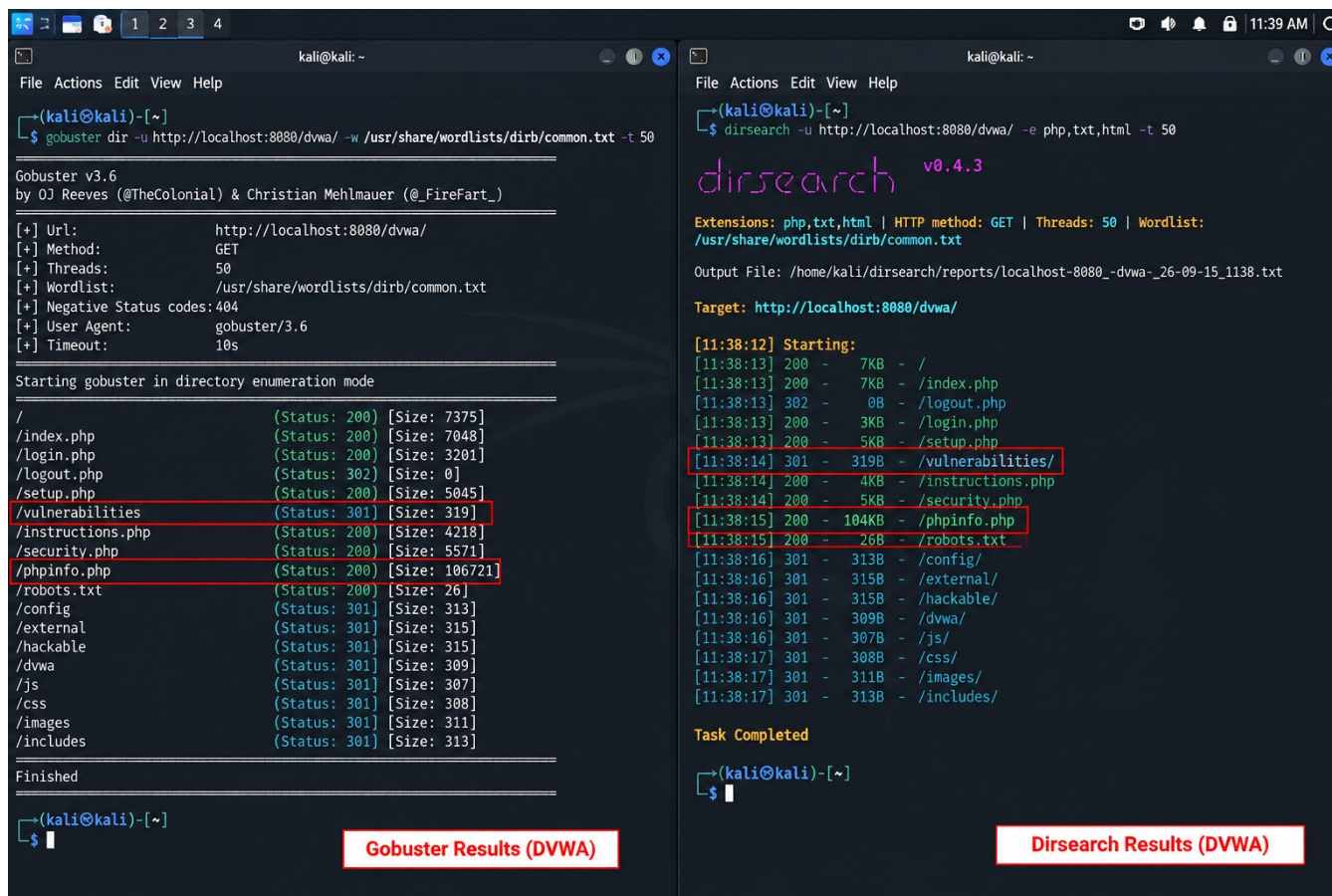
11. Task 25 — Directory & Endpoint Discovery

Not every application endpoint is linked from the visible interface. Directory and endpoint discovery attempts to identify paths that may be hidden, forgotten, administrative, or intended for APIs. This can improve coverage during a security assessment.

Burp can support manual discovery through Site Map, HTTP history, links, redirects, JavaScript references, robots.txt, and observed application behavior. Gobuster and Dirsearch can complement this process by requesting candidate paths from a wordlist.

Discovery results require validation. A path returning HTTP 200 does not automatically represent a vulnerability; the tester must determine what the endpoint does and whether authentication and authorization are correctly enforced.

GOBUSTER / DIRSEARCH RESULTS



The image displays two terminal windows side-by-side, showing the results of directory and endpoint discovery tools. The left window shows Gobuster v3.6 results, and the right window shows Dirsearch v0.4.3 results. Both tools were used to scan the target http://localhost:8080/dvwa/ using a wordlist located at /usr/share/wordlists/dirb/common.txt. The Gobuster results list various endpoints with their status codes and sizes. The Dirsearch results show a list of discovered paths with their status codes and sizes. Red boxes highlight specific results in both windows.

```
(kali@kali)~$ gobuster dir -u http://localhost:8080/dvwa/ -w /usr/share/wordlists/dirb/common.txt -t 50

Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@_FireFart_)

[+] Url: http://localhost:8080/dvwa/
[+] Method: GET
[+] Threads: 50
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.6
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

/ (Status: 200) [Size: 7375]
/index.php (Status: 200) [Size: 7048]
/login.php (Status: 200) [Size: 3201]
/logout.php (Status: 302) [Size: 0]
/setup.php (Status: 200) [Size: 5045]
/vulnerabilities (Status: 301) [Size: 319]
/instructions.php (Status: 200) [Size: 4218]
/security.php (Status: 200) [Size: 5571]
/phpinfo.php (Status: 200) [Size: 106721]
/robots.txt (Status: 200) [Size: 26]
/config (Status: 301) [Size: 313]
/external (Status: 301) [Size: 315]
/hackable (Status: 301) [Size: 315]
/dvwa (Status: 301) [Size: 309]
/js (Status: 301) [Size: 307]
/css (Status: 301) [Size: 308]
/images (Status: 301) [Size: 311]
/includes (Status: 301) [Size: 313]

Finished

(kali@kali)~$
```

Gobuster Results (DVWA)

```
(kali@kali)~$ dirsearch -u http://localhost:8080/dvwa/ -e php,txt,html -t 50

dirsearch v0.4.3

Extensions: php,txt,html | HTTP method: GET | Threads: 50 | Wordlist: /usr/share/wordlists/dirb/common.txt
Output File: /home/kali/dirsearch/reports/localhost-8080_-dvwa_-26-09-15_1138.txt
Target: http://localhost:8080/dvwa/

[11:38:12] Starting:
[11:38:13] 200 - 7KB - /
[11:38:13] 200 - 7KB - /index.php
[11:38:13] 302 - 0B - /logout.php
[11:38:13] 200 - 3KB - /login.php
[11:38:13] 200 - 5KB - /setup.php
[11:38:14] 301 - 319B - /vulnerabilities/
[11:38:14] 200 - 4KB - /instructions.php
[11:38:14] 200 - 5KB - /security.php
[11:38:15] 200 - 104KB - /phpinfo.php
[11:38:15] 200 - 26B - /robots.txt
[11:38:16] 301 - 313B - /config/
[11:38:16] 301 - 315B - /external/
[11:38:16] 301 - 315B - /hackable/
[11:38:16] 301 - 309B - /dvwa/
[11:38:16] 301 - 307B - /js/
[11:38:17] 301 - 308B - /css/
[11:38:17] 301 - 311B - /images/
[11:38:17] 301 - 313B - /includes/

Task Completed

(kali@kali)~$
```

Dirsearch Results (DVWA)

12. Vulnerability Finding — SQL Injection

Description

The SQL Injection weakness occurs when user-controlled input is incorporated into a database query without safe parameterization. The DVWA SQL Injection module intentionally demonstrates this vulnerability, allowing a controlled test input to alter the expected query behavior.

Root Cause

The underlying security design problem is unsafe construction of SQL statements from untrusted input. String concatenation or equivalent query construction can allow input to become part of SQL syntax.

Steps to Reproduce

8. Open the authorized DVWA SQL Injection module.
9. Submit a normal identifier and record the baseline response.
10. Repeat the request using the controlled test input.
11. Observe whether additional records or altered query behavior appear.
12. Capture the request and response in Burp.

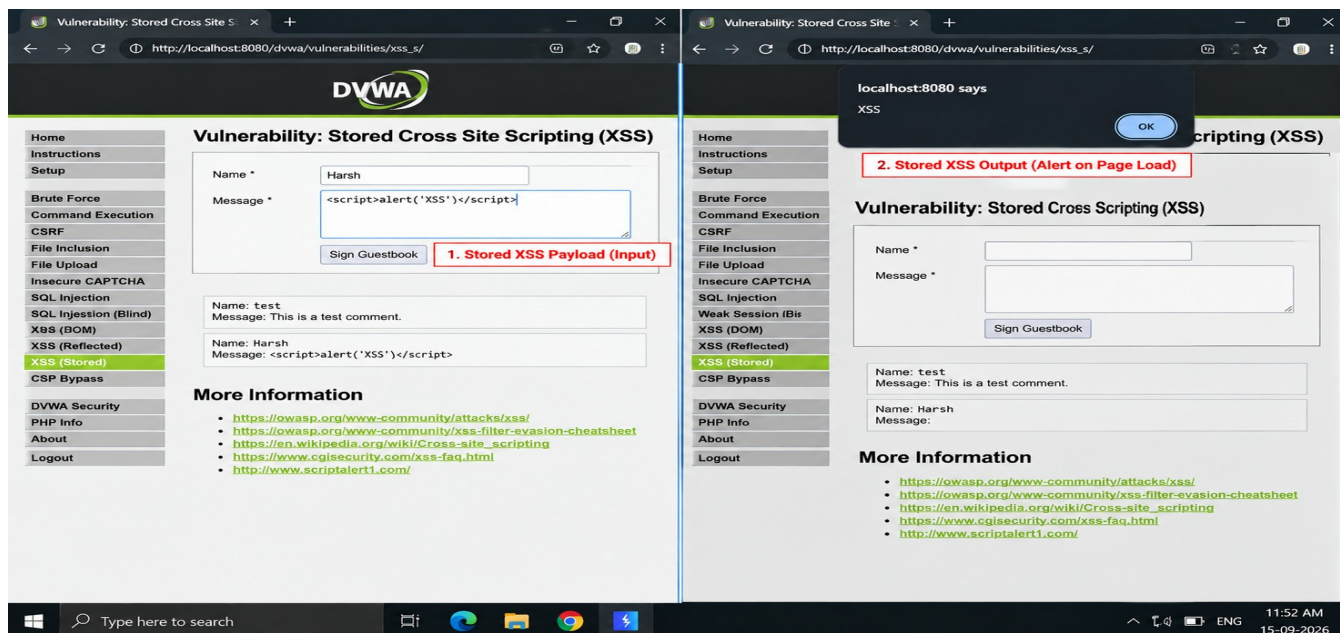
Impact

Depending on database permissions and application design, SQL Injection may permit unauthorized reading or modification of database content and can sometimes enable further compromise.

Remediation

- Use parameterized queries / prepared statements.
- Validate input types and reject unexpected formats.
- Use a least-privileged database account.
- Do not expose raw database errors to users.
- Add automated security tests for injection-prone endpoints.

SQL INJECTION PROOF



13. Vulnerability Finding — Stored XSS

Description

The Stored XSS vulnerability allows attacker-controlled content to be saved by the application and later rendered in a user's browser without appropriate output encoding. In DVWA, the guestbook/comment functionality provides an intentional demonstration.

Root Cause

The application fails to treat stored content as untrusted data at the point where it is rendered. Input validation alone is not sufficient; output must be safely encoded for its specific context.

Steps to Reproduce

13. Open the DVWA Stored XSS module.
14. Enter the harmless XSS proof-of-concept.
15. Submit the message.
16. Reload or revisit the page containing the stored entry.
17. Observe the harmless JavaScript alert.
18. Capture the stored input and execution result.

Impact

Stored XSS can affect users who view the compromised content. Depending on application privileges and browser context, it may facilitate unauthorized actions, content manipulation, phishing, or other client-side abuse.

Remediation

- Apply contextual output encoding before rendering stored data.
- Use safe templating and DOM APIs.

- Implement an appropriate Content Security Policy.
- Apply secure session cookie attributes.
- Validate stored data according to business requirements.

STORED XSS PROOF

Untitled Session - OWASP ZAP 2.15.0

File Edit View Analyse Report Tools Import Online Help

Standard Mode

Sites

- http://localhost:8080
 - dvwa
 - css
 - images
 - includes
 - js
 - login.php
 - logout.php
 - security.php
 - setup.php
 - vulnerabilities
 - brute
 - exec
 - file
 - sql
 - xss

Quick Start → Request × → Response +

GET http://localhost:8080/dvwa/index.php

Header: Text Body: Text

GET /dvwa/index.php HTTP/1.1
Host: localhost:8080
User-Agent: OWASP ZAP/2.15.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive

HTTP/1.1 200 OK
Date: Mon, 15 Sep 2026 06:12:14 GMT
Server: Apache/2.4.58 (Win64) OpenSSL/3.1.3 PHP/8.2.12
X-Powered-By: PHP/8.2.12
Content-Length: 7342
Content-Type: text/html; charset=UTF-8

<!DOCTYPE html>
<html lang="en-GB">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
<title>Welcome :: Damn Vulnerable Web Application (DVWA)</title>

History Search Alerts Output Spider Active Scan Fuzzer AJAX Spider

Alerts (12)

- Cross Site Scripting (Reflected) (3)
- Cross Site Scripting (Stored) (2)
- SQL Injection (2)
- Directory Browsing (1)
- Information Disclosure - PHP Errors (1)
- Cookie No HttpOnly Flag (1)
- Cookie Without Secure Flag (1)
- X-Content-Type-Options Header Missing (1)
- X-Frame-Options Header Missing (1)
- Content Security Policy (CSP) Header Not Set (1)
- Server Leaks Information via "X-Powered-By" Header (1)
- Timestamp Disclosure (1)

Cross Site Scripting (Reflected)

Risk: High
Confidence: High
Parameter: name
Attack: <script>alert(1)</script>
Evidence: <script>alert(1)</script>
CWE ID: 79
WASC ID: 8
Source: Active (40012 - Cross Site Scripting (Reflected))

Description:
Cross Site Scripting (XSS) occurs when untrusted data is included in a web page without proper validation or escaping, allowing an attacker to execute arbitrary script in the context of the victim's browser.

Other Info:
The vulnerable parameter was found in a GET request and the payload was reflected in the response.

URL:
http://localhost:8080/dvwa/vulnerabilities/xss_r/
Method:
GET

Alerts 12 0 8 4 0 Current Scans 0 0 0 0 0 0

14. Vulnerability Finding — Reflected XSS

Description

Reflected XSS occurs when untrusted request data is returned to the browser and interpreted as executable content. The DVWA reflected XSS module provides a controlled environment for demonstrating this behavior.

Root Cause

The application reflects user-controlled input into a response without applying context-appropriate output encoding or otherwise preventing execution.

Steps to Reproduce

19. Open the authorized DVWA Reflected XSS module.
20. Submit the harmless XSS proof-of-concept.
21. Observe whether the input is reflected in the resulting page.
22. Confirm the harmless alert in the local browser.
23. Capture the request and resulting output.

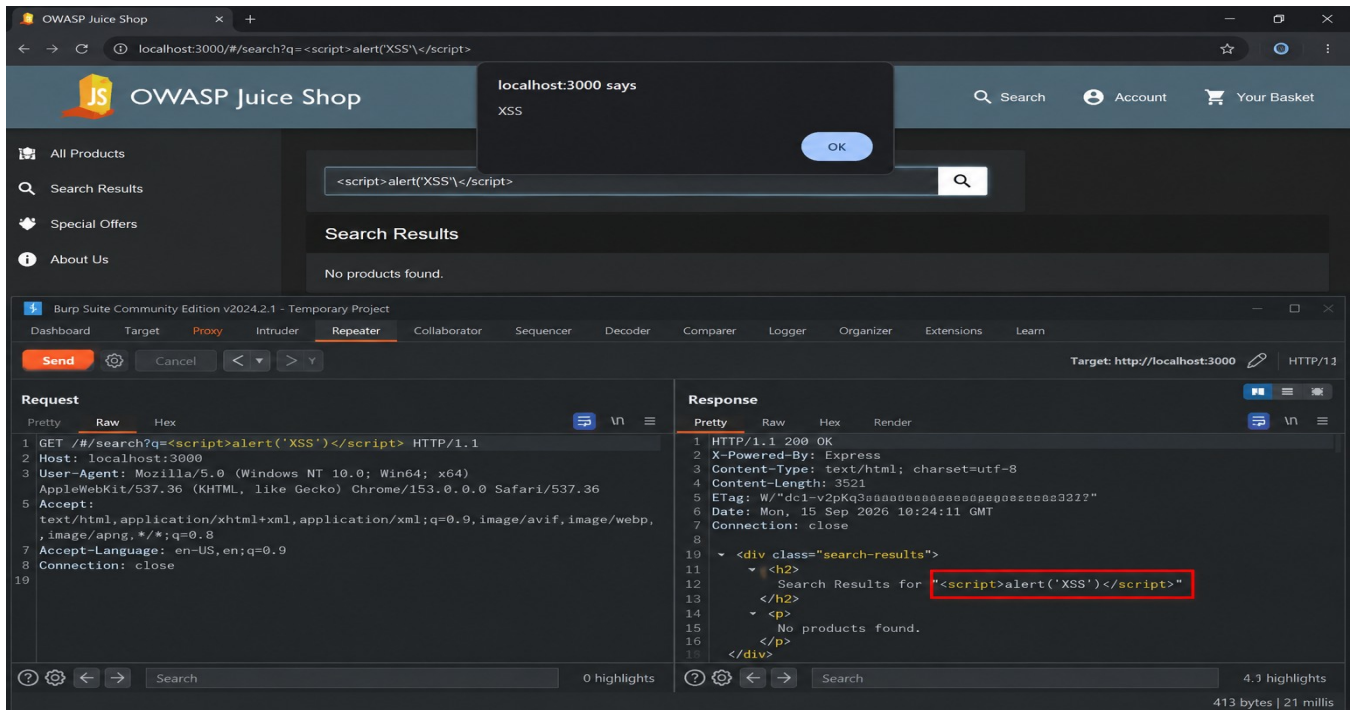
Impact

An attacker could craft a malicious link or request that causes script execution when a victim interacts with it. The practical impact depends on the page context, victim privileges, and browser protections.

Remediation

- Encode output according to the context in which it is rendered.
- Validate expected input and reject inappropriate content where appropriate.
- Use a restrictive Content Security Policy as defense in depth.
- Review client-side code for unsafe DOM sinks.

REFLECTED XSS PROOF



15. Supporting Validation — OWASP ZAP

OWASP ZAP can provide automated spidering and vulnerability alerts that complement manual Burp testing. Automated scanners can increase coverage and identify potential issues that deserve manual review. However, scanner results can contain false positives, duplicates, or informational observations.

A professional assessment therefore treats scanner output as supporting evidence. A finding should be manually reviewed and, where possible, reproduced before it is presented as a confirmed vulnerability.

OWASP ZAP ALERTS PANEL

The screenshot displays the OWASP ZAP application interface. The top menu bar includes File, Edit, View, Analyse, Report, Tools, Import, Online, and Help. The main workspace is divided into three panes: Sites, Request, and Response.

Sites Pane: Shows a tree view of the target site `http://localhost`. The `dvwa` directory is expanded, showing subdirectories like `GET: index.php`, `GET: login.php`, `GET: security.php`, and `vulnerabilities`. The `vulnerabilities` directory is further expanded, showing files like `GET: sql.php`, `POST: sql.php`, `GET: xss_r.php`, `POST: xss_r.php`, `GET: file_upload.php`, `POST: file_upload.php`, and `GET: dvwaPage.php`.

Request Pane: Shows the raw request for `GET /dvwa/vulnerabilities/sql.php?id=1&Submit=Submit HTTP/1.1`. The request headers include `Host: localhost`, `User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)`, `Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8`, `Accept-Language: en-US,en;q=0.9`, `Accept-Encoding: gzip, deflate, br`, `Connection: keep-alive`, and `Upgrade-Insecure-Requests: 1`.

Response Pane: Shows the raw response for `HTTP/1.1 200 OK`. The response headers include `Date: Sat, 20 Sep 2026 10:25:11 GMT`, `Server: Apache/2.4.58 (Win64) PHP/8.2.12`, `X-Powered-By: PHP/8.2.12`, `Content-Length: 3412`, and `Content-Type: text/html; charset=UTF-8`. The response body contains HTML code, including a `<title>Vulnerability: SQL Injection :: Damn Vulnerable Web App` and a form with an input field named `id` and a submit button.

Alerts Panel: The bottom pane shows a list of alerts. The `SQL Injection (2)` alert is selected, showing details:

- Risk:** High
- Confidence:** High
- Parameter:** id
- Attack:** `1' OR '1'='1`
- Evidence:** `1' OR '1'='1`
- CWE ID:** 89
- WASC ID:** 19
- Source:** Active Scan
- Description:** SQL injection may be possible.
- Other Info:** The application appears to be vulnerable to SQL injection via the id parameter.
- Solution:** Sanitize input using prepared statements or parameterized queries.

The bottom status bar shows `Alerts 9` and `Current Scans 0`.

EXPORTED ZAP REPORT

1

OWASP ZAP Scan Report

2

Cross Site Scripting (Reflected)

3

Cross Site Scripting (Stored)

4

SQL Injection

OWASP ZAP Scan Report

Target

Scan Type

Generated

ZAP Version

Report Format

User

Description

http://localhost:8080/dvwa/

Active Scan

2026-09-15 12:05:42

2.15.0

HTML

Harsh

Security scan report for DVWA (Damn Vulnerable Web Application) using OWASP ZAP.

Summary of Alerts

Risk Level	Number of Alerts	Percentage
High	4	22%
Medium	6	33%
Low	6	33%
Informational	2	11%
Total	18	100%

Alert Distribution

High

Medium

Low

Informational

Top 5 Alerts

#	Risk	Alert	Instances
1	High	Cross Site Scripting (Reflected)	3
2	High	Cross Site Scripting (Stored)	2
3	Medium	SQL Injection	2
4	Medium	Directory Browsing	1
5	Low	Information Disclosure - PHP Errors	1

16. Risk Summary & Remediation Plan

Finding	Security Risk	Primary Remediation
SQL Injection	Database query manipulation and potential unauthorized data access	Prepared statements + least-privilege DB access
Stored XSS	Script execution for users viewing stored content	Contextual output encoding + CSP + safe rendering
Reflected XSS	Script execution through crafted request/link	Contextual output encoding + input validation
Parameter Tampering	Manipulation of client-controlled application values	Server-side authorization and business-rule validation
Endpoint Exposure	Unnecessary or insufficiently protected application paths	Remove unused endpoints + enforce access controls

16.1 Defense-in-Depth Principles

- Never trust client-controlled data.
- Validate authorization on every sensitive server-side operation.
- Use secure defaults and least privilege.
- Separate authentication from authorization decisions.
- Use parameterized database access.
- Encode output according to its context.
- Protect sessions and cookies.
- Log security-relevant events without exposing sensitive data.
- Perform security testing again after remediation.

17. Assessment Limitations

This assessment is an educational exercise performed against intentionally vulnerable training software. Results are representative of the configured laboratory and should not be generalized to production systems without separate testing.

- Testing was limited to the authorized local environment.
- Only selected web vulnerability categories were assessed.
- No destructive testing or denial-of-service activity was performed.
- No real customer data or third-party accounts were targeted.
- Automated scanner output was not treated as independently conclusive.
- Severity classifications are contextual and should be refined using the final evidence.

18. Conclusion

The Week 7 VAPT exercise provides a practical foundation for web application security testing. By combining reconnaissance, HTTP interception, manual request manipulation, vulnerability validation, endpoint discovery, and structured reporting, the tester develops a workflow that is closer to professional application-security assessment than simple tool usage.

Burp Suite provides visibility into the communication between browser and server, while Repeater and Intruder allow controlled experimentation with application inputs. The assessment also demonstrates why vulnerabilities should be supported by reproducible evidence rather than identified solely by automated tools.

The documented findings — SQL Injection, Stored XSS, and Reflected XSS — illustrate common failures in input handling, database query construction, and output encoding. Applying the recommended controls, followed by retesting, can substantially improve the security posture of an application.